

AgentHive - Complete Tech Stack Explained in Simple Language

Version: 1.0

Project: AgentHive

Purpose: Explain every technology used in AgentHive in a simple, practical, and professional way.

1. What is AgentHive?

AgentHive is a platform where we can create and manage multiple AI agents from one dashboard.

A normal chatbot only answers one message at a time.

AgentHive is different.

It can have multiple agents, such as:

- Personal Assistant Agent
- Developer Agent
- PDF Agent
- Research Agent
- Task Agent
- Workflow Agent
- Reminder Agent

Each agent can do a specific job.

Simple example:

User says:

Read this PDF, summarize it, and create tasks from it.

AgentHive flow:

1. User uploads PDF.
2. Dashboard sends request to backend.
3. Backend gives work to Document Agent.
4. Document Agent reads the file.
5. LLM Router selects Gemini or fallback model.
6. AI summarizes the PDF.
7. Task Agent creates tasks.

8. Database saves result.
9. Logs record everything.
10. Dashboard shows final answer.

In simple words:

AgentHive is like an office where different AI employees work together.

2. Common Example Used Throughout This Document

To understand the full project, we will use one common example.

User request:

Summarize this project PDF and create 5 action items.

Now we will see how every technology helps in this one request.

3. Full Request Flow in AgentHive

Simple flow:

```
User
↓
Frontend Dashboard
↓
Nginx
↓
FastAPI Backend
↓
Agent Orchestrator
↓
Document Agent
↓
LLM Router
↓
Gemini / Ollama / HuggingFace / GPT
↓
PostgreSQL + pgvector
↓
```

```
Redis + Celery
↓
Logs + Monitoring
↓
Final Response
```

Meaning:

The user does not directly talk to database, AI model, Redis, or workers.

The user only uses the dashboard.

Everything else happens behind the scenes.

4. Docker

What is Docker?

Docker is used to run every part of the project inside separate containers.

A container is like a small isolated box.

Each box has one responsibility.

Example:

- One box for frontend
- One box for backend
- One box for database
- One box for Redis
- One box for Ollama
- One box for Grafana

Simple Explanation

Imagine you are shifting a restaurant to another city.

Without Docker:

You need to manually arrange:

- Gas
- Stove

- Ingredients
- Tables
- Staff
- Menu
- Lights

With Docker:

Everything is packed into ready-made boxes.

You open the boxes and the restaurant starts working.

Why We Use Docker in AgentHive

AgentHive has many services.

Without Docker, every developer needs to install manually:

- Node.js
- Python
- PostgreSQL
- Redis
- Ollama
- Grafana
- Prometheus
- Loki
- MinIO

That is difficult and error-prone.

With Docker:

One command starts everything:

```
docker compose up
```

How Docker Helps AgentHive

Benefits:

- Same setup for every developer
- Easy local development
- Easy production deployment
- Services are isolated
- Easy to restart one service

- Easy to scale later

Example in AgentHive

If backend crashes, only backend container restarts.

Frontend, database, Redis, and Grafana keep running.

That makes the system more stable.

5. Docker Compose

What is Docker Compose?

Docker Compose is used to run multiple Docker containers together.

Docker runs one container.

Docker Compose runs the full project.

Simple Explanation

Docker is one vehicle.

Docker Compose is traffic management for all vehicles.

It knows:

- Which container should start first
- Which container depends on which
- Which ports to open
- Which environment variables to use
- Which volumes to attach

Why We Use Docker Compose

AgentHive needs many services:

```
frontend
backend
postgres
redis
worker
```

```
scheduler  
ollama  
minio  
nginx  
grafana  
prometheus  
loki  
promtail
```

Starting all manually is painful.

Docker Compose starts all together.

Example

Development command:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up
```

Production command:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

How It Helps

- Easy startup
- Easy shutdown
- Easy logs
- Easy service management
- Easy local testing

6. Frontend - Next.js

What is Next.js?

Next.js is the frontend framework used to build the AgentHive dashboard.

The dashboard is what the user sees.

Simple Explanation

If AgentHive is a car, Next.js is the dashboard and steering wheel.

The user clicks buttons, fills forms, sees logs, and manages agents from there.

Why We Use Next.js

Next.js is good for:

- Dashboards
- Routing
- Fast UI
- API integration
- Modern React development
- Production-ready frontend apps

What We Build With Next.js

Pages:

```
/dashboard  
/agents  
/agents/create  
/models  
/workflows  
/logs  
/settings  
/env  
/monitoring
```

Example

User opens:

```
http://localhost:3000/agents
```

They can see all agents:

- Developer Agent
- PDF Agent
- Research Agent
- Task Agent

They can enable, disable, edit, or run an agent.

Technical Explanation

Next.js is a React-based framework that supports:

- File-based routing
- Server-side rendering
- Client components
- API communication
- Production builds

How It Helps AgentHive

It gives us a clean professional dashboard to control the whole platform.

7. React

What is React?

React is a JavaScript library used to build UI components.

Simple Explanation

Think of the dashboard as made of Lego blocks.

Each UI part is one block.

Examples:

- Button
- Sidebar
- Table
- Agent Card
- Log Viewer
- Model Selector

React helps us build and reuse these blocks.

Why We Use React

Because dashboard UI has many reusable parts.

Example:

The same status badge can be used for:

- Agent status
- Model status
- Workflow status
- Job status

Example

Agent status badge:

```
Enabled  
Disabled  
Running  
Failed
```

Instead of writing the same UI again and again, we create one component.

How It Helps

- Faster development
 - Reusable UI
 - Cleaner code
 - Better user experience
-

8. Tailwind CSS

What is Tailwind CSS?

Tailwind CSS is used to style the frontend quickly.

Simple Explanation

Normally, to design a button, we write separate CSS.

With Tailwind, we style directly using small utility classes.

Example

Instead of writing large CSS files, we can write:

```
<button className="px-4 py-2 rounded-lg bg-black text-white">  
  Run Agent  
</button>
```

Why We Use It

AgentHive needs a professional dashboard.

Tailwind helps us create:

- Cards
- Tables
- Forms
- Sidebar
- Responsive layouts
- Dark mode
- Status badges

How It Helps

- Fast UI development
 - Consistent design
 - Less CSS mess
 - Easy customization
-

9. Nginx

What is Nginx?

Nginx is a reverse proxy and gateway.

Simple Explanation

Think of Nginx as a receptionist at a big office.

Visitors come to the receptionist.

The receptionist sends them to the right department.

In AgentHive:

- Frontend requests go to Next.js

- API requests go to FastAPI
- Monitoring requests go to Grafana

The user does not need to know where every service is running.

What is a Reverse Proxy?

A reverse proxy sits in front of backend services.

User talks to Nginx.

Nginx talks to internal services.

Simple example:

User:

```
agenthive.com/api/agents
```

Nginx sends it to:

```
backend:8000
```

User never sees backend directly.

Why We Use Nginx

We use it for:

- Routing
- Security
- Single entry point
- SSL support
- Future load balancing
- Cleaner production setup

Example in AgentHive

Request:

```
agenthive.com/dashboard
```

Nginx sends to frontend.

Request:

```
agenthive.com/api/agents
```

Nginx sends to backend.

Request:

```
agenthive.com/monitoring
```

Nginx sends to Grafana.

What If We Do Not Use Nginx?

Users need different URLs:

```
Frontend: localhost:3000  
Backend: localhost:8000  
Grafana: localhost:3001  
MinIO: localhost:9001
```

That is not clean.

Nginx makes everything professional.

10. Backend - FastAPI

What is FastAPI?

FastAPI is the Python backend framework used in AgentHive.

Backend is the brain that handles requests.

Simple Explanation

Frontend is what user sees.

Backend is what does the actual work.

When user clicks:

Run Agent

Frontend sends request to backend.

Backend decides what to do.

Why We Use FastAPI

FastAPI is good for AI systems because:

- It is Python-based
- It is fast
- It supports async
- It has automatic API docs
- It works well with AI libraries
- It is easy to structure cleanly

Example in AgentHive

API:

```
POST /api/agents/run
```

This API receives:

- Agent ID
- User request
- Selected model
- Tools allowed

Then backend starts the agent run.

Technical Use

FastAPI handles:

- Agent APIs
- Model APIs
- Workflow APIs
- Logs APIs
- Settings APIs
- File upload APIs
- Health check APIs

How It Helps

FastAPI becomes the central control system of AgentHive.

11. PostgreSQL

What is PostgreSQL?

PostgreSQL is the main database.

Simple Explanation

PostgreSQL is like AgentHive's permanent notebook.

Anything important goes there.

What We Store

AgentHive stores:

- Users
- Agents
- Model settings
- Workflow settings
- Logs
- Agent runs
- Tool calls
- Files metadata
- Prompt versions
- System settings

Example

When user creates a new agent:

Name:

Developer Agent

Model:

Gemini

Tools:

GitHub Tool, Code Tool

This data is saved in PostgreSQL.

Why We Use PostgreSQL

PostgreSQL is:

- Free
- Open-source
- Reliable
- Powerful
- Production-ready
- Good for structured data

How It Helps

Without database, AgentHive forgets everything after restart.

With PostgreSQL, all settings remain saved.

12. pgvector

What is pgvector?

pgvector is an extension for PostgreSQL that stores vector embeddings.

Simple Explanation

Normal database search is like searching exact words.

Vector search is like searching meaning.

Example

Stored text:

Docker helps package applications into containers.

User asks:

How do we make the app run the same everywhere?

Normal search may fail because words are different.

Vector search understands meaning and finds Docker-related content.

Why We Use pgvector

AI agents need memory.

They should remember:

- Past conversations
- Documents
- User preferences
- Project knowledge
- Previous agent results

pgvector helps search memory by meaning.

How It Helps AgentHive

PDF Agent can answer questions from uploaded documents.

Developer Agent can remember project architecture.

Research Agent can reuse previous research.

13. Redis

What is Redis?

Redis is a very fast in-memory storage system.

Simple Explanation

PostgreSQL is like a permanent notebook.

Redis is like a sticky note on your desk.

It is used for temporary and fast data.

Why We Use Redis

We use Redis for:

- Cache
- Queue broker
- Temporary session data
- Rate limiting
- Fast lookup
- Celery task messaging

Example

User asks the same question twice.

First time:

AI model is called.

Second time:

Redis can return cached answer.

This saves:

- Time
- Tokens
- API cost

Another Example

User uploads a large PDF.

Backend does not process it immediately inside the request.

It puts a job in Redis queue.

Worker picks it later.

How It Helps

Redis makes AgentHive faster and more scalable.

14. Celery

What is Celery?

Celery is a background job system.

Simple Explanation

Imagine a restaurant.

Waiter takes order.

Kitchen prepares food.

The waiter does not cook.

In AgentHive:

FastAPI receives request.

Celery worker does long work.

Why We Use Celery

Some tasks take time:

- PDF processing
- Research
- Long workflows
- Email scanning
- Embedding generation
- File indexing

If backend waits for all this, user experience becomes slow.

With Celery:

Backend quickly says:

Job started

Worker processes in background.

Example

User uploads 200-page PDF.

Flow:

1. FastAPI receives file.
2. Job added to Redis.
3. Celery worker processes PDF.
4. Result saved in database.
5. Dashboard shows status.

How It Helps

- Avoids blocking API
- Handles long jobs
- Supports retries
- Can run multiple workers
- Scales easily

15. Celery Beat / Scheduler

What is Scheduler?

Scheduler runs tasks at fixed times.

Simple Explanation

It is like an alarm clock.

It tells AgentHive:

Run this task every day at 9 AM.

Why We Use It

For scheduled agents.

Examples:

- Daily email summary
- Weekly report generation
- Daily backup
- Reminder checks
- Model health check

Example

Every morning at 8 AM:

Personal Assistant Agent runs.

It checks:

- Tasks
- Calendar
- Pending work

Then creates daily summary.

16. MinIO

What is MinIO?

MinIO is self-hosted file storage.

It works like AWS S3 but runs locally.

Simple Explanation

PostgreSQL stores information about files.

MinIO stores the actual files.

Example:

PostgreSQL stores:

```
file_name = project.pdf  
file_id = 123  
file_type = pdf
```

MinIO stores:

```
actual project.pdf file
```

Why We Use MinIO

AgentHive needs file uploads:

- PDFs
- Images
- Documents
- Reports
- Logs export
- Generated files

Using MinIO avoids paid AWS storage during development.

Example

User uploads PDF.

File goes to MinIO.

Metadata goes to PostgreSQL.

PDF Agent reads file from MinIO.

How It Helps

- Free local storage
 - S3-compatible
 - Easy future AWS migration
 - Keeps database clean
-

17. Ollama

What is Ollama?

Ollama lets us run AI models locally.

Simple Explanation

Normally AI model runs on cloud.

With Ollama, AI model runs on our own machine.

Why We Use Ollama

Because AgentHive is free-first.

If Gemini quota finishes, we can use Ollama.

Example

Primary model:

Gemini

If Gemini fails:

Fallback to Ollama

Models We Can Run

Examples:

- Llama
- Mistral
- Qwen
- DeepSeek
- Phi

How It Helps

- No API cost
- Works locally

- Private
- Good fallback
- Useful during development

Limitation

Ollama depends on local machine power.

If laptop is slow, model may respond slowly.

18. Gemini

What is Gemini?

Gemini is Google's AI model provider.

Why We Use Gemini

AgentHive uses Gemini as default primary model because:

- Good quality
- Large context support
- Free tier available
- Good for general tasks

Simple Explanation

Gemini is like the main expert AI brain.

Whenever agent needs thinking, it first asks Gemini.

Example

User asks:

Summarize this PDF.

PDF Agent extracts text.

LLM Router sends text to Gemini.

Gemini returns summary.

What If Gemini Fails?

Fallback system starts.

Example:

```
Gemini quota exceeded
↓
Use Ollama
```

19. OpenAI / GPT

What is GPT?

GPT is another AI model provider.

Why We Keep GPT Support

Even if not used by default, AgentHive should support it.

Reason:

Some tasks may need stronger reasoning or better coding support.

Example

Developer Agent may use GPT only when enabled.

Dashboard setting:

```
GPT fallback: disabled by default
```

If user enables it:

```
Fallback order: Gemini → Ollama → GPT
```

How It Helps

- More provider flexibility

- No vendor lock-in
 - Better fallback choices
-

20. Hugging Face

What is Hugging Face?

Hugging Face provides many open-source AI models.

Simple Explanation

It is like a library of AI models.

We can use models for:

- Text generation
- Embeddings
- Classification
- Summarization

Why We Use It

AgentHive can use Hugging Face as:

- Optional LLM provider
- Embedding provider
- Model experimentation platform

Example

If Gemini is unavailable and Ollama is slow, Hugging Face can be another fallback.

21. Groq

What is Groq?

Groq is an AI inference provider known for fast responses.

Why We Support It

For fast LLM responses when free/cheap access is available.

Example

Research Agent needs quick short summary.

Groq can be selected from dashboard as fast model provider.

22. LLM Router

What is LLM Router?

LLM Router is AgentHive's model decision system.

Simple Explanation

Imagine you have multiple teachers:

- Gemini teacher
- Ollama teacher
- GPT teacher
- Hugging Face teacher

LLM Router decides which teacher should answer.

Why We Need It

Agents should not directly call Gemini or GPT.

If every agent directly calls a model, changing model becomes difficult.

Instead:

Agent asks LLM Router.

Router chooses model.

Example

Policy:

```
Primary: Gemini
Secondary: Ollama
Fallback: Hugging Face
```

User request comes.

Router checks:

1. Is Gemini enabled?
2. Is token limit available?
3. Is Gemini healthy?
4. If yes, use Gemini.
5. If no, use Ollama.
6. If Ollama fails, use Hugging Face.

How It Helps

- No hardcoded model
- Easy fallback
- Dashboard control
- Cost control
- Token control
- Provider independence

23. Agent Orchestrator

What is Agent Orchestrator?

Agent Orchestrator decides which agent should work.

Simple Explanation

It is like a manager in a company.

User is the boss.

Agents are employees.

Orchestrator is the manager who gives work to the right employee.

Example

User says:

Summarize this PDF.

Orchestrator selects:

Document Agent

User says:

Find bugs in this code.

Orchestrator selects:

Developer Agent

User says:

Plan my day.

Orchestrator may call:

- Calendar Agent
- Task Agent
- Reminder Agent

Why We Need It

Without orchestrator, user must manually choose everything.

With orchestrator, system becomes intelligent.

24. Agent Registry

What is Agent Registry?

Agent Registry stores all available agents.

Simple Explanation

It is like an employee directory.

It knows:

- Agent name
- Agent role
- Allowed tools
- Default model
- Status
- Version

Example

Developer Agent:

```
Role: Code review and debugging
Tools: Code tool, GitHub tool
Model: Gemini
Fallback: Ollama
Status: Enabled
```

Why We Use It

So the dashboard can manage agents dynamically.

No need to hardcode agents.

25. Workflow Engine

What is Workflow Engine?

Workflow Engine connects multiple steps together.

Simple Explanation

It is like a recipe.

Example recipe:

1. Read email
2. Extract tasks
3. Add tasks
4. Send reminder

Example in AgentHive

Workflow:

```
PDF uploaded
↓
Document Agent summarizes
↓
Task Agent creates tasks
↓
Notification Agent informs user
```

Why We Use It

Many real-world tasks need multiple agents.

Workflow Engine makes this automatic.

26. TOON Format

What is TOON?

TOON is a compact way to send structured data to LLMs.

Simple Explanation

JSON is detailed but token-heavy.

TOON is shorter.

Example JSON

```
{  
  "agent": "developer",  
  "task": "review_code",  
  "priority": "high"  
}
```

Compact TOON Style

```
agent:developer  
task:review_code  
priority:high
```

Why We Use It

LLMs charge or limit based on tokens.

Less text means:

- Lower cost
- Faster response
- More context space

Where We Use TOON

Use TOON for:

- LLM prompts
- Tool result summaries
- Agent memory context
- Workflow context

Do not use TOON for normal frontend-backend API.

Use JSON for APIs.

27. Professional Logging

What is Logging?

Logging means recording what happened in the system.

Simple Explanation

It is like CCTV for software.

If something goes wrong, we can see what happened.

What We Log

- User request
- Agent selected
- Model used
- Token usage
- Tool calls
- Errors
- Fallback events
- Workflow steps
- Response time

Example

```
User requested PDF summary
Document Agent started
Gemini called
Gemini failed due to quota
Ollama fallback used
Summary generated
Response returned
```

Why We Need It

Without logs:

We guess what went wrong.

With logs:

We know exactly what happened.

28. Grafana

What is Grafana?

Grafana is a dashboard for monitoring.

Simple Explanation

Grafana is like the control room of AgentHive.

It shows health of the system.

Why We Use Grafana

We want to see:

- Is backend healthy?
- Is Redis working?
- Is Postgres slow?
- Are agents failing?
- Is token usage high?
- Are workers overloaded?

Example

If many agents fail, Grafana chart shows error spike.

If Redis queue is growing, Grafana shows workers are overloaded.

Why Local Grafana?

Because we want free monitoring during development.

No cloud monitoring cost.

How It Helps

- Better debugging
- Better production visibility
- Faster issue detection
- Professional monitoring

29. Prometheus

What is Prometheus?

Prometheus collects metrics.

Simple Explanation

Grafana shows charts.

Prometheus collects numbers for those charts.

Examples of Metrics

- API requests per minute
- Agent runs per hour
- Failed agent runs
- Average response time
- Redis queue size
- Token usage
- CPU usage
- Memory usage

Why We Use It

Without Prometheus, Grafana has no data.

Prometheus is the data collector.

Grafana is the visual dashboard.

30. Loki

What is Loki?

Loki stores logs.

Simple Explanation

Prometheus stores numbers.

Loki stores text logs.

Example

Prometheus tells:

Backend errors increased.

Loki tells:

Exact error message and where it happened.

Why We Use Loki

To search logs from:

- Backend
- Worker
- Scheduler
- Docker containers
- Agent runs

How It Helps

When Agent fails, we can search by:

```
agent_run_id  
request_id  
user_id  
model_provider
```

31. Promtail

What is Promtail?

Promtail collects logs and sends them to Loki.

Simple Explanation

Promtail is like a delivery person.

It picks logs from containers and delivers them to Loki.

Flow

```
Backend logs
↓
Promtail
↓
Loki
↓
Grafana
```

Why We Use It

Without Promtail, Loki will not automatically receive logs.

32. OpenTelemetry

What is OpenTelemetry?

OpenTelemetry tracks request journey across services.

Simple Explanation

Imagine tracking a parcel.

You can see:

- Packed
- Shipped
- Reached warehouse
- Out for delivery
- Delivered

OpenTelemetry does the same for requests.

Example

User runs agent.

Trace:

```
Frontend request
↓
Backend API
↓
Agent Orchestrator
↓
LLM Router
↓
Gemini
↓
Database save
↓
Response
```

If request is slow, we can see where it was slow.

Why We Use It

Logs tell what happened.

Metrics tell how much.

Tracing tells where time was spent.

33. Environment Variables

What are Environment Variables?

Environment variables store configuration outside code.

Simple Explanation

Instead of writing secrets inside code, we keep them in `.env`.

Example

```
DATABASE_URL=postgresql://agenthive:agenthive@postgres:5432/agenthive
GEMINI_API_KEY=your_key_here
OLLAMA_BASE_URL=http://ollama:11434
```

Why We Use Them

Different environments need different settings.

Local:

```
localhost database
```

Production:

```
server database
```

Code remains same.

Only env changes.

Security Rule

Never commit real secrets to GitHub.

Use:

```
.env.example
```

for sample values.

34. GitHub Actions / CI-CD

What is CI/CD?

CI/CD means automatically testing and deploying code.

Simple Explanation

When developer pushes code, system automatically checks:

- Is code valid?
- Do tests pass?
- Can Docker build?
- Can it deploy?

Why We Use It

To avoid broken code reaching production.

Example Flow

```
Developer pushes code
↓
GitHub Actions runs tests
↓
Docker image builds
↓
Deployment starts
```

How It Helps

- Fewer manual mistakes
 - Faster deployment
 - Better code quality
 - Professional workflow
-

35. Security Layer

What Security Means Here

AgentHive handles:

- API keys
- User data
- Files
- Agent actions
- Logs

So security is important.

What We Protect

- API keys
- Environment variables
- User files
- Model provider credentials
- Admin settings

Simple Example

Dashboard should show:

```
GEMINI_API_KEY=****abcd
```

Not full key.

Security Features

- Mask secrets
 - Auth checks
 - Role-based access later
 - Audit logs
 - Secure env handling
 - No secrets in logs
 - File validation
-

36. Monitoring vs Logging vs Tracing

These three are different.

Logging

What happened?

Example:

```
Gemini failed because quota exceeded.
```

Metrics

How often or how much?

Example:

```
Gemini failed 20 times today.
```


Tracing

Where did time go?

Example:

```
Request took 8 seconds because LLM call took 7 seconds.
```

Together they give full visibility.

37. How One PDF Request Uses Everything

User request:

```
Summarize this PDF and create 5 action items.
```

Full flow:

1. User uploads PDF from Next.js dashboard.
2. Nginx routes request to FastAPI.
3. FastAPI validates file.
4. MinIO stores PDF.
5. PostgreSQL stores file metadata.
6. Redis receives background job.
7. Celery worker starts processing.
8. Document Agent extracts PDF text.
9. Text is chunked and compressed.
10. pgvector stores searchable memory.
11. LLM Router selects Gemini.
12. If Gemini fails, Ollama fallback is used.
13. AI summary is generated.
14. Task Agent creates action items.
15. PostgreSQL stores final result.
16. Logs are sent to Loki.
17. Metrics are collected by Prometheus.
18. Grafana shows run status.
19. Dashboard displays final output.

Simple meaning:

The user only clicks upload.

AgentHive handles everything else.

38. Why This Tech Stack is Good

Free First

Most services are open-source:

- Docker
- PostgreSQL
- Redis
- Celery
- MinIO
- Ollama
- Grafana
- Prometheus
- Loki

Scalable

Later we can add:

- More backend instances
- More workers
- More agents
- More models
- More users

Flexible

We are not locked to one AI provider.

We can use:

- Gemini
- Ollama
- GPT
- Hugging Face
- Groq

Professional

The system has:

- Logging
 - Monitoring
 - Docker
 - CI/CD
 - Clear docs
 - Modular backend
 - Dashboard control
-

39. Final Simple Explanation

AgentHive is like a smart company.

User is the boss.

Dashboard is the office reception.

Nginx is the building gate.

FastAPI is the operations team.

Agent Orchestrator is the manager.

Agents are employees.

LLM Router chooses the best expert.

Gemini, Ollama, GPT are expert brains.

PostgreSQL is the permanent record room.

pgvector is intelligent memory.

Redis is temporary fast memory.

Celery is background staff.

MinIO is file storage room.

Grafana is control room.

Prometheus collects health numbers.

Loki stores event history.

Promtail delivers logs.

Docker keeps every department in separate boxes.

Together, they create a professional multi-agent AI platform.